Unsupervised Multivariate Time Series Anomaly Detection by Feature Decoupling in Federated Learning Scenarios

Yifan He, Xi Ding, Yateng Tang, Jihong Guan, and Shuigeng Zhou

***Abstract*—Anomalies are usually regarded as data errors or novel patterns previously unseen, which are quite different from most observed data. Accurate detection of anomalies is crucial in various application scenarios. This article focuses on unsupervised anomaly detection from multivariate time series (MTS), as real-world data collected from sources such as wearable devices, medical equipment, and industrial machines typically manifest as MTS and are often unlabeled. Anomaly detection in MTS represents a data-driven challenge that traditionally requires substantial centralized data for training models. However, in practice, data are frequently distributed among multiple institutions, with privacy concerns restricting unrestricted access. To address these issues, we introduce feature decoupling federated learning (FDFL), an approach designed to collaboratively train a representation learning network over multiple clients for unsupervised anomaly detection in MTS. Unlike previous methods that simply integrate MTS anomaly detection (MTS-AD) algorithms with federated learning (FL) strategies, FDFL specifically addresses heterogeneity among clients by decoupling the representation network into shared and private branches through a contrastive learning mechanism. This method aggregates shared parameters during each federated round while maintaining client-specific private parameters locally. Additionally, we develop a self-attention block that integrates the representations derived from both shared and private parameters to reconstruct MTS and identify anomalies based on reconstruction errors. Extensive experiments conducted on three publicly available datasets demonstrate that FDFL outperforms existing algorithms in most cases, highlighting the effectiveness and superiority of our proposed method in MTS-AD.**

***Impact Statement*—In fields such as finance and healthcare, stringent data privacy, and security regulations significantly restrict data sharing among institutions, thus spurring the development of FL. While recent research has begun to explore MTS-AD under the FL frameworks, most existing approaches rely on supervised learning and task-specific networks, essentially integrating anomaly detection algorithms into existing FL architectures. However, in unsupervised MTS-AD scenarios, where labeled data are unavailable, distribution drift becomes ubiquitous due to variations in devices, collection methodologies, and target objects, with noisy data commonly from edge devices. This article introduces feature-decoupling federated anomaly detection (FDFL), a novel method tailored for unsupervised MTS data. FDFL addresses these challenges by decoupling each client's network into shared and private branches, leveraging heterogeneity-robust and personalized loss functions. The model is trained through contrastive learning, which enhances its ability to adapt to diverse data sources while mitigating the effect of distribution drift. In doing so, FDFL facilitates the more effective utilization of MTS data from various sources in an unsupervised setting, ensuring robust performance even in the presence of noisy data and distributional changes.**

***Index Terms*—Feature decoupling, federated learning (FL), multivariate time series (MTS), unsupervised anomaly detection.**

I. Introduction

MODERN intelligent devices, such as sensors, continuously gather multivariate time series (MTS) data from a variety of individuals or systems. Automating anomaly detection from long-term monitoring data is crucial for managing and maintaining the stability of monitored objects. In MTS data, anomalies constitute only a small fraction, and obtaining accurate anomaly labels is often challenging. Consequently, unsupervised MTS-AD has emerged as an increasingly important area of data mining research. Over the past decades, various unsupervised MTS-AD methods have been proposed. These approaches typically model normal patterns using density estimation, clustering, or reconstruction, and identify the instances that deviate significantly from these patterns as anomalies. Most existing methods assume centralized storage of training data. However, practical limits such as privacy concerns, regulatory restrictions, and high transmission cost make it infeasible to transfer data from edge devices to a central server for processing [1]. Therefore, there is a pressing need to develop anomaly detection algorithms capable of collaboratively processing MTS data distributed across multiple edge devices while safeguarding privacy and data security.

Federated learning (FL) [2] is a paradigm that retains training data distributed across edge devices or systems (referred to as clients) while collaboratively learning a shared model by

Received 20 September 2024; revised 20 November 2024 and 21 December 2024; accepted 19 January 2025. Date of publication 14 February 2025; date of current version 31 July 2025. This article was recommended for publication by Associate Editor Anand Nayyar upon evaluation of the reviewers' comments. *(Corresponding author: Shuigeng Zhou.)*

Yifan He, Xi Ding, and Shuigeng Zhou are with School of Computer Science, Fudan University, Shanghai 200438, China and also with Shanghai Key Laboratory of Intelligent Information Processing, Shanghai 200438, China (e-mail: yfhe20@fudan.edu.cn; dingx22@m.fudan.edu.cn; sgzhou@fudan.edu.cn).

Yateng Tang is with Tencent Weixin Group, Guangzhou 510308, China (e-mail: fredyttang@tencent.com).

Jihong Guan is with the Department of Computer Science and Technology, Tongji University, Shanghai 201804, China, and also with the Key Laboratory of Embedded System and Service Computing, Tongji University, Ministry of Education, Shanghai 200092, China (e-mail: jhguan@tongji.edu.cn).

Digital Object Identifier 10.1109/TAI.2025.3533437

aggregating locally computed updates. This paradigm has wide applications in various domains, including financial risk analysis, medical image processing [3], and Internet of Things (IoT) management [4], where it enables the aggregation of decentralized data while mitigating privacy and security concerns. Some recent studies [5], [6], [7] have explored time series anomaly detection under the FL framework. However, most of them perform FL in a supervised manner, focus more on developing particular representation networks for specific tasks and simply integrate anomaly detection algorithms with existing federated frameworks.

In unsupervised MTS anomaly detection, no labeled data are available to guide the learning process. Additionally, data collected from edge devices are often noisy, and distribution drift is common due to variations in device characteristics, acquisition tools, and target objects. Liu et al. [8] introduced FedTADBench, a benchmark for federated unsupervised MTS-AD, which encompasses five representative MTS-AD algorithms and four popular FL mechanisms. Although FedTADBench incorporates regularization terms or control variates in its federated frameworks to mitigate the "client-drift" problem, it does not adequately address the reduction of noise and heterogeneity in complex distributed MTS data from the perspective of time series representation. This limitation can lead to suboptimal performance.

To address the aforementioned challenges, this article introduces feature decoupling federated learning (FDFL) for unsupervised MTS anomaly detection. FDFL decouples each client's network into a shared branch and a private branch by employing a contrastive training approach with heterogeneity robust loss (HRL) and personalized loss (PL). The shared branch is designed to extract client-invariant representations, thereby enhancing robustness against noise and heterogeneity across different clients. This branch focuses on capturing information critical to anomaly detection, such as temporal trend changes and abrupt peak values. To achieve this, we introduce noise disturbance and downsampling to each original MTS sequence and generate a corresponding rescaled MTS sequence. By optimizing the HRL between the representations of the original and rescaled sequences (both extracted by the shared branch), the shared branch can effectively counteract noise and statistical heterogeneity while emphasizing key anomaly indicators. The private branch, on the other hand, captures personalized information specific to each client, serving as a supplement to the shared branch. We guide this branch to focus on local characteristics unique to each client by minimizing the PL loss, thus enhancing its complementarity to the shared branch. Furthermore, we incorporate a self-attention mechanism to integrate the representations from both the shared and private branches for MTS reconstruction. Joint training among clients is conducted by optimizing the reconstruction loss, HRL, and PL simultaneously. Anomalies are identified based on the reconstruction errors. Importantly, the shared network facilitates communication across clients during each federated round, while the private networks preserve local privacy consistently.

Contributions of this article are summarized as follows.

1) *Feature Decoupling Framework for Unsupervised MTS-AD*: We introduce a novel feature decoupling-based approach for the detection of unsupervised MTS anomalies under the FL framework. This method disentangles the network into shared and private branches, aggregating only the shared branch during each communication round to ensure privacy-preserving collaboration.
2) *Innovative Loss Functions for Feature Decoupling*: We design two specialized loss functions to achieve effective feature decoupling: HRL enables the shared branch to focus on client-invariant representations, making it robust against noise and client drift. PL allows the private branch to capture personalized features unique to each client, complementing the shared branch by representing information not captured in the invariant features.
3) *Empirical Validation of the FDFL Approach*: We perform extensive experiments on three publicly available MTS datasets, demonstrating the effectiveness and superiority of the proposed FDFL approach.

The rest of the article is organized as follows. Section II reviews the related work. The problem statement is presented in Section III, and details of our proposed method FDFL are given in Section IV. Section V introduces the evaluation results of the proposed method. Finally, Section VI summarizes the article and highlights future work.

II. Related Work

In this section, we survey the related work from three aspects: unsupervised MTS-AD, FL, and feature decoupling.

A. Unsupervised MTS-AD

Unsupervised MTS-AD is a widely studied problem in academia and industry. The core concept involves modeling typical patterns and identifying deviations from these norms as anomalies. Classical methods have detected anomalies based on density estimation. Local outlier factor (LOF) [9] introduces the concept of local reachable density (LRD), classifying instances with high LRD as anomalies. Deep autoencoding Gaussian Mixture model (DAGMM) [10] uses the Gaussian Mixture model to estimate the density of representations. Some works define normal patterns as clusters, then anomaly detection is considered as the one-class classification [11] problem. Support vector data description (SVDD) [12] and DeepSVDD [13] obtain the cluster centers from training data and then determine whether an instance is an anomaly by its distance to the cluster centers. Temporal hierarchical one-class network (THOC) [14] employs a hierarchical recurrent neural network (RNN) to extract representations of MTS data, implements a hierarchical clustering mechanism, and calculates the multilayer distances between intermediate representations and hierarchical cluster centers to determine anomaly scores.

Most existing methods rely on reconstruction, assuming that normal instances should be reconstructed accurately, while anomalies exhibit significant reconstruction errors. Various deep-learning networks are utilized to extract hidden representations of MTS data and reconstruct them [15]. Autoencoder [16] is a well-known network for feature representation and reconstruction. To address the sequential nature of MTS data, long short-term memory (LSTM)-VAE [17] integrates LSTM

with variational autoencoder (VAE) technology to capture temporal features and reconstruct the data. OmniAnomaly [18] introduces normalizing flow to LSTM-VAE and uses reconstruction probability for detection. InterFusion [19] models the normal pattern inside MTS data through hierarchical VAE with two stochastic latent variables. Transformer [20] is a powerful network based solely on attention mechanism, which achieves excellent representations of sequential data. TranAD [21] is a transformer-based autoencoder for anomaly detection and diagnosis, it leverages self-conditioning and adversarial training to amplify errors and gain training stability. Anomaly Transformer [22] leverages a transformer framework for data representation and introduces the association discrepancy (AssDis) metric to enhance the distinction between normal and anomalous instances.

In addition to capturing the sequential characteristics of time series, representations of MTS data should also account for the interdependencies among multiple variates [23]. Graph deviation network (GDN) [24] incorporates graph neural networks (GNNs) to model dependencies among multiple variates and identifies anomalies based on the prediction errors of these variates. Deep variational graph convolutional recurrent network (DVGCRN) [25] models channel dependencies and stochastic elements within MTS through an embedding-guided probabilistic generative network, which is integrated with VGCRN to capture both spatial and temporal fine-grained correlations in MTS data. These approaches have demonstrated effectiveness in both MTS representation and anomaly detection.

However, no algorithm dominates all the other approaches and fits all anomaly detection setups [26]. Given that anomaly detection is inherently data-driven, increasing the volume of training data typically results in improved performance. Aggregating data from multiple sources to model normal patterns represents a promising avenue for future research.

B. FL

FL is a machine learning paradigm where many clients collaboratively train a model under the orchestration of a central server while keeping the training data decentralized [27]. Based on the distribution of data across multiple participants, FL can be categorized into three types: horizontal FL, vertical FL, and federated transfer learning. This study focuses on horizontal FL, as the data features remain consistent across clients in our MTS-AD task. FedAvg [2] is a widely adopted horizontal FL method that updates parameters using stochastic gradient descent (SGD) on randomly selected clients and aggregates these parameters following several local training rounds. To address the statistical heterogeneity intrinsic to federated networks, FedProx [28] introduces a proximal term to accommodate nonidentically distributed data across clients. Scaffold [29] utilizes control variates to mitigate distributional shifts within local clients. Moon [30] applies contrastive learning at the model level to enhance performance on non-IID data distributions.

Since anomaly detection is a data-driven problem, more training data could model normal patterns better and achieve more accurate anomaly detection. Several studies have explored anomaly detection under the FL framework. FadMan [31] introduces a pioneering algorithm for federated anomaly detection across multiple attributed networks. FS-G [32] offers a unified, comprehensive, and efficient package for federated graph learning, with applications in real-world e-commerce scenarios. Cavallin and Mayer [5] investigated the application of FL for supervised, semisupervised, and unsupervised anomaly detection. Specifically for time series data, Zhang et al. [33] developed ConvGRU-VAE along with its federated framework to tackle the challenges of entity-level anomaly detection. Mothukuri et al. [6] proposed a GRU-based FL approach for supervised anomaly detection in IoT networks using decentralized on-device data. Liu et al. [34] designed attention mechanism-based convolutional neural network LSTM for IoT anomaly detection and training models in a federated way. DeepFed [7] is a FL framework tailored for multiple industrial cyber–physical systems (CPSs), facilitating the collaborative development of comprehensive intrusion detection models while preserving privacy. FedTADBench [8] serves as a benchmark incorporating five representative time series anomaly detection algorithms alongside four popular FL methods.

However, the aforementioned approaches primarily focus on designing temporal feature extraction networks tailored for specific tasks and directly integrate these networks with existing FL frameworks. Addressing the challenges of noise and heterogeneity among clients in unsupervised MTS-AD requires exploring specialized designs for both the MTS representation network and the FL framework.

C. Feature Decoupling

Feature decoupling, also known as disentangled representation learning, aims to decompose embeddings into multiple independent components to improve model interpretability and robustness. This approach has been applied in computer vision [35], [36], speech signal processing [37], and recommendation systems [38]. VAE-based methods are widely used for time series feature decoupling. Mathieu et al. [39] leveraged both VAE [40] and GAN [41] to build a deep conditional generative model that learns to separate the factors of variation associated with the labels from the other sources of variability. FHVAE introduces the factorized hierarchical VAE, which learns disentangled and interpretable representations for sequence-level and segment-level attributes without supervision. Li et al. [42] devised a minimalist generative model for learning disentangled representations of high-dimensional time series, incorporating a global latent variable for content features and a stochastic RNN with time-local latent variables for dynamic features. FactorVAE [43] achieves feature decoupling by encouraging the distribution of representations to be factorial, thereby ensuring independence across dimensions. S3VAE [44] designs self-supervised auxiliary tasks to learn disentangled time-invariant and time-varying representations for time series data. To ensure that the two sets of representations are decoupled and represent distinct data characteristics, Tonekeboni et al. [45] introduced counterfactual regularization that aims at minimizing mutual information to achieve disentanglement. To improve the accuracy of MTS prediction, D3TN [46] proposes a disentangled dynamic

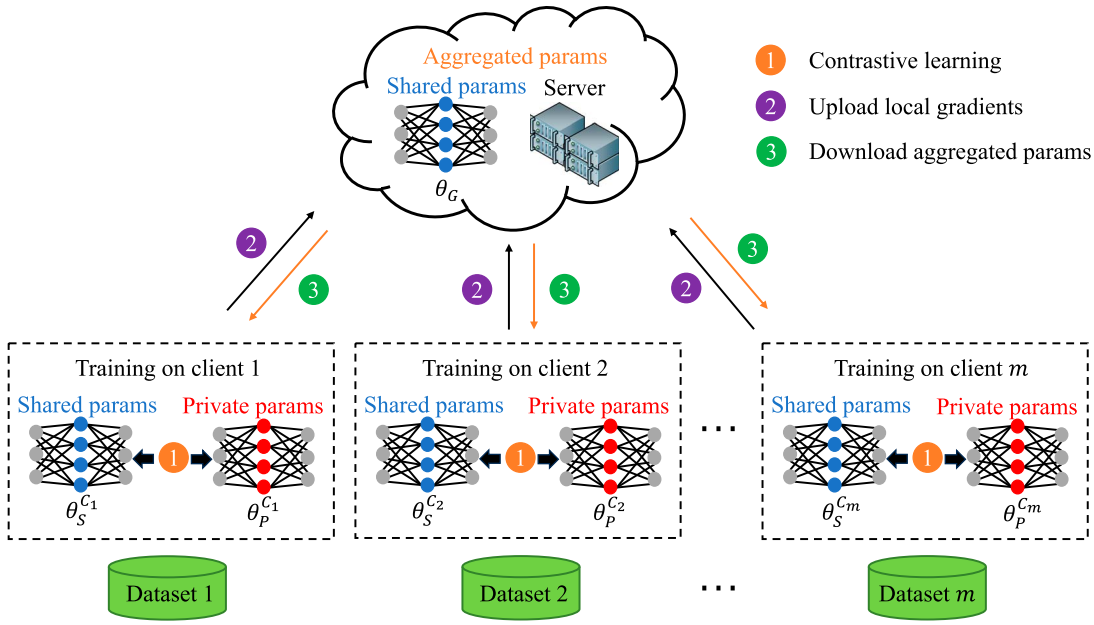

Fig. 1. Framework of our FDFL method.

deviation transformer network to jointly exploit multiscale dynamic intersensor relationships and long-term temporal dependencies.

Existing feature decoupling methods focus mainly on extracting independent components from the data to generate more interpretable and robust representations, thereby enhancing the performance of downstream tasks. In contrast, in this article, our goal is to decouple features into client-invariant and client-dependent components.

III. Problem Formulation

In this study, we focus on detecting anomalies from MTS at the sequence level. The MTS data contain successive observations, represented as $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T]$, where $\mathbf{x}_t \in \mathbb{R}^N$ represents the observation at time t, N is the number of variates. Given a set of n MTS sequences $\mathcal{X} = \{\mathbf{X}_1, \mathbf{X}_2, \ldots, \mathbf{X}_n\}$, the goal of sequence level MTS-AD is to determine which sequence $\mathbf{X}_i$ is an anomaly.

In practical applications, concerns over privacy breaches lead to a situation where direct data exchange between clients is prohibited, thereby creating "data silos" that will limit the effectiveness of unsupervised anomaly detection models. Recently, FL has emerged as a solution to address the challenge of data silos without compromising on privacy. In a typical FL framework, the private data of each client is not shared, and the clients are jointly trained by aggregating the intermediate parameters of the client models, thus achieving the balance between privacy preservation and model performance. However, for unsupervised anomaly detection, simply sharing parameters between clients may overlook the uniqueness of each client, resulting in performance degradation. To deal with this, we introduce a novel feature decoupling approach within the FL paradigm for unsupervised multitime series (MTS) anomaly detection. This method bifurcates the MTS representation network into shared and private branches, enabling the sharing of common parameters among clients while retaining unique parameters exclusively for use by their respective local clients.

IV. Methodology

A. Overview

In this article, we propose a FDFL method for anomaly detection from decentralized MTS data over a number of clients, as shown in Fig. 1. The clients do not share data and communicate with each other for privacy preservation, and the server coordinates the clients to complete the model training collaboratively. Given the prevalent domain shifts inherent with the decentralized data sources, our goal is to identify common patterns across clients while preserving their individual characteristics. We achieve this by decoupling the features of MTS data into shared and private components. The shared component captures commonalities across clients, whereas the private component retains client-specific information, which are implemented by dedicated shared and private branches, respectively, within our network architecture. During training, the server aggregates the shared features from all clients and redistributes the aggregated results back to them. Meanwhile, clients retain their private features locally and integrate the aggregated shared features received from the server. This iterative process facilitates continuous improvement and adaptation of the model, which is detailed in Section IV-C.

Like most unsupervised anomaly detection methods, our FDFL is a reconstruction-based method that optimizes model parameters and identifies anomalies according to the reconstruction error. In order to effectively represent MTS data and supervise feature decoupling, we build a double-stream LSTM autoencoder network for MTS reconstruction and introduce a

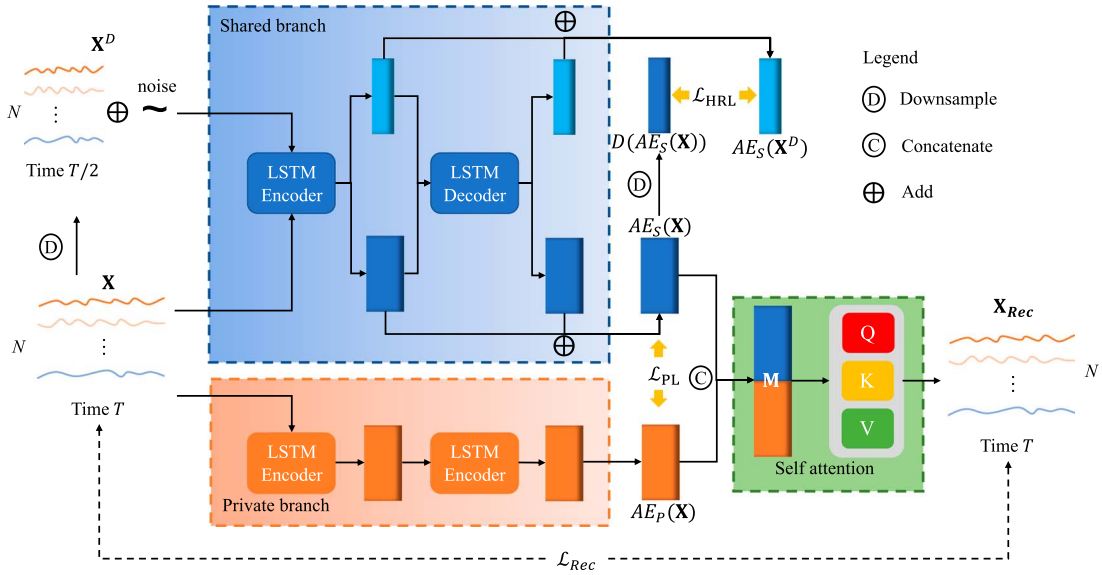

Fig. 2. Network structure of each client.

contrastive learning approach to enhance the robustness of the shared branch while making each private branch a comprehensive complement to the shared branch. As illustrated in Fig. 2, for each MTS sequence, we create an augmented counterpart by introducing noise and applying downsampling. We then reduce the discrepancy between the representations of the original and augmented sequences, thereby enhancing the shared branch's resilience to noise and data variability. Furthermore, we minimize the similarity between the representations of the shared branch and each private branch, which makes each private branch focus on the features that the shared branch ignores. We describe the details of the network structure of each client in Sections IV-D and IV-E. As a summary, we outline the whole training and inference processes of our method in an algorithm in Section IV-F.

B. Data Preprocessing

Each variate of MTS often has different scales. To stabilize the gradient descent step and make the model converge robust and faster. We perform normalization as follows:

$$\mathbf{x}_t = \frac{\mathbf{x}_t - \min(\mathbf{X})}{\max(\mathbf{X}) - \min(\mathbf{X}) + \epsilon} \tag{1}$$

where $\max(\mathbf{X})$ and $\min(\mathbf{X})$ denote the maximum and minimum vectors of MTS $\mathbf{X}$, ϵ is a small constant vector, the operations "+," "−," and "/" are evaluated in an element wise way. After normalization, the MTS data are rescaled to the range $[0, 1)$. As the maximum and minimum vectors also contain privacy information, the normalization of each client is done independently.

C. Federated Parameter Aggregation

FL aims to collaboratively train a shared global model without centralizing or sharing raw training data. However, data drift among decentralized clients is common, leading to performance degradation over time. In this article, we propose a novel federated framework that decouples each client's parameters into shared and private components, as illustrated in Fig. 1. In each communication round, the shared parameters of local clients are initialized with the latest global weights and then fine-tuned locally on their respective datasets. Meanwhile, private parameters are trained using a contrastive learning approach. Consistent with many anomaly detection methods, our approach identifies anomalies based on reconstruction loss. The loss function of each client is as follows:

$$\mathcal{L}_{\text{Rec}} = \frac{1}{n}\sum_{i}^{n} |\mathbf{X}_i - \mathbf{X}_{i\text{Rec}}| \tag{2}$$

where $\mathbf{X}_{i\text{Rec}}$ is derived from both shared parameters θ_S and private parameters θ_P, i.e.,

$$\mathbf{X}_{i\text{Rec}} = f(\theta_S, \theta_P, \mathbf{X}_i). \tag{3}$$

The details of reconstruction and feature decoupling will be elaborated in the subsequent subsections. The learned shared parameters $\theta_S^{C_j}$ are aggregated to a new global model: $\theta_G \leftarrow \sum_{j=1}^{m} w_j \theta_S^{C_j}$, where $w_j = (n_j / \sum_{j=1}^{m} n_j)$ is the respective weight coefficient calculated based on n_j—the number of MTS sequences on each client.

D. Reconstruction Network

The local model structure of each client is shown in Fig. 2, which includes both shared parameters and private parameters that are used to reconstruct the original time series data. We utilize LSTM to construct the autoencoders of the shared and private branches for MTS reconstruction. LSTM is a well-known recurrent network containing forget gates, which can solve gradient disappearance and gradient explosion when training models on long sequences.

LSTM extracts the representations of sequence data by three gates: forget gate, update gate, and output gate. For a given

time series $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T]$, the input to LSTM at step t contains the current data $\mathbf{x}_t$, the hidden state vector $\mathbf{h}_{t-1}$, and the cell state vector $\mathbf{c}_{t-1}$ of step $(t-1)$. The forget gate selectively forgets $\mathbf{c}_{t-1}$ while the update gate memorizes the critical information of $\mathbf{x}_t$. The output gate scales the current cell state $\mathbf{c}_t$ by the tanh activation function and outputs the hidden state vector h_t. The process is formulated as follows:

$$\begin{aligned} \mathbf{c}_t &= \mathbf{z}^f \odot \mathbf{c}_{t-1} + \mathbf{z}^u \odot \mathbf{z} \\ \mathbf{h}_t &= \mathbf{z}^o \odot \tanh(\mathbf{c}_t) \end{aligned} \tag{4}$$

where $\mathbf{z}^f$, $\mathbf{z}^u$, and $\mathbf{z}^o$ are the activation vectors of the forget gate, update gate, and output gate, respectively. $\mathbf{z}$ is the cell activation vector and $\odot$ denotes the elementwise product. All activation vectors could be derived from $\mathbf{x}_t$ and $\mathbf{h}_{t-1}$:

$$\begin{aligned} \mathbf{z}^f &= \sigma(\mathbf{W}^f \odot \text{concat}(\mathbf{x}_t, \mathbf{h}_{t-1})) \\ \mathbf{z}^u &= \sigma(\mathbf{W}^u \odot \text{concat}(\mathbf{x}_t, \mathbf{h}_{t-1})) \\ \mathbf{z}^o &= \sigma(\mathbf{W}^o \odot \text{concat}(\mathbf{x}_t, \mathbf{h}_{t-1})) \\ \mathbf{z} &= \tanh(\mathbf{W} \odot \text{concat}(\mathbf{x}_t, \mathbf{h}_{t-1})) \end{aligned} \tag{5}$$

where σ denotes the sigmoid activation function, $\mathbf{W}$, $\mathbf{W}^f$, $\mathbf{W}^u$ and $\mathbf{W}^o$ are the weight matrices that include bias vector. For each LSTM block in our model, we initialize $\mathbf{h}_0$ and $\mathbf{c}_0$ as zero vectors, and output the hidden state vectors $\mathbf{H} \in \mathbb{R}^{T \times d}$ of all steps by

$$\mathbf{H} = \text{LSTM}(\mathbf{X}, \mathbf{h}_0, \mathbf{c}_0). \tag{6}$$

To extract more informative representations of MTS, we add a shortcut connection from the hidden layer to the output layer of the AE structure. We formulate the AE as follows:

$$\text{AE}(\mathbf{X}) = \text{LSTM}(\mathbf{X}) \oplus \text{LSTM}(\text{LSTM}(\mathbf{X})) \tag{7}$$

where $\oplus$ represents the elementwise add. In unsupervised MTS-AD, there is not any supervision information to guide weight assignments to the shared and private representations. Therefore, we first concatenate the shared representation $\text{AE}_S(\mathbf{X}) \in \mathbb{R}^{T \times d}$ with the private representation $\text{AE}_P(\mathbf{X}) \in \mathbb{R}^{T \times d}$ into a representation matrix $\mathbf{M} \in \mathbb{R}^{T \times 2d}$ (AE_S and AE_P are the autoencoder networks of the shared branch and private branch) and then build an attention network to assign weight to each dimension. The attention mechanism produces output based on the input Queries, Keys, and Values. The output is computed as the weighted sum of these values where Queries and the corresponding Keys compute the weight assigned to each Value. In this work, we use a self-attention mechanism that generates Queries, Keys, and Values using only $\mathbf{M}$, i.e., $\mathbf{Q} = \mathbf{M}\mathbf{W}^q$, $\mathbf{K} = \mathbf{M}\mathbf{W}^k$, $\mathbf{V} = \mathbf{M}\mathbf{W}^v$, where $\mathbf{Q}$, $\mathbf{K} \in \mathbb{R}^{T \times d_k}$, and $\mathbf{V} \in \mathbb{R}^{T \times d_v}$ are the feature matrices of Queries, Keys, and Values, $\mathbf{W}^q, \mathbf{W}^k \in \mathbb{R}^{2d \times d_k}$, and $\mathbf{W}^v \in \mathbb{R}^{2d \times d_v}$ are parameter matrices. Then, we use scaled dot-product to compute the output of attention as follows:

$$\begin{aligned} SA(\mathbf{M}) &= f(\mathbf{W}^q, \mathbf{W}^k, \mathbf{W}^v, M) \\ &= \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V. \end{aligned} \tag{8}$$

To counteract the effect of small gradients caused by big d_k, we set $(1/\sqrt{d_k})$ as a scaling factor and use a softmax function to assign weights to Values. After getting the fusion representation of the shared and private branches, we use a feed-forward neural network to reconstruct MTS data from the fusion representation

$$\mathbf{X}_{\text{Rec}} = \mathbf{W}^r SA(\mathbf{M}) + \mathbf{b}^r \tag{9}$$

where $\mathbf{W}^r$ and $\mathbf{b}^r$ are the weight matrix and bias vector, respectively.

E. Feature Decoupling

Owing to variations in data collection equipment and sources across different clients, distribution drift is a pervasive issue. To mitigate the statistical heterogeneity while leveraging the shared temporal dependency features of the distributed clients, we propose to disentangle the model parameters θ into shared θ_S and private θ_P, and only share the former in the FL process. To further enforce the feature decoupling, we introduce two losses: 1) *HRL*: The shared branch should be robust to the heterogeneity of the data. We add a tiny noise perturbation to the original MTS data and downsample the data with a stride of 2. The representation of the rescaled MTS sequence should be similar to that of the original MTS data. 2) *PL*: The latent representations of the shared branch and private branch should be orthogonal to each other, which ensures the uniqueness of each local client. Specifically, the two losses above are defined as follows:

$$\mathcal{L}_{\text{HRL}} = \frac{1}{n}\sum_{i}^{n} \text{sim}(D(\text{AE}_S(\mathbf{X_i})), \text{AE}_S(\mathbf{X_i^D})) \tag{10}$$

$$\mathcal{L}_{\text{PL}} = \frac{1}{n}\sum_{i}^{n} (1 - \text{sim}(\text{AE}_S(\mathbf{X_i}), \text{AE}_P(\mathbf{X_i}))) \tag{11}$$

where $D(\cdot)$ denotes downsample operation, sim represents the cosine similarity function, and $\mathbf{X_i^D}$ is the MTS sequence downsampled from $\mathbf{X_i}$. The overall objective function is as follows:

$$\mathcal{L} = \mathcal{L}_{\text{Rec}} + \alpha\mathcal{L}_{\text{HRL}} + \beta\mathcal{L}_{\text{PL}} \tag{12}$$

where α and β are two hyperparameters.

F. Algorithm

The training and inference processes of our FDFL method are summarized in Algorithm 1. First, we initialize the parameters of shared and private branches for each client. We select several clients to participate in the model parameter training and aggregation in each federated round (line 3). Then, we jointly minimize $\mathcal{L}_{\text{Rec}}$, $\mathcal{L}_{\text{HRL}}$, and $\mathcal{L}_{\text{PL}}$ to optimize the local model, which can be done in parallel (lines 7–12). Finally, we aggregate shared branches of selected clients and jump to the next federated round until convergence (line 14). In the inference stage, we calculate the MTS sequences' anomaly scores in each client by calculating the reconstruction errors (lines 18–24).

Algorithm 1: FDFL: Feature Decoupling Federated Learning for Unsupervised MTS Anomaly Detection.

Require: Initial shared θ_G, $\theta_G = \{AE_S, SA, \mathbf{W}^r\}$
Ensure: Initial private $\theta_P^{C_i}$ for each client C_i, $\theta_P^{C_i} = AE_P^{C_i}$

```
# Training Stage
for each round r = 1, ..., R do
    sample clients S ⊆ {C_1, ..., C_m}
    communicate θ_G to all clients i ∈ S
    for client C_i ∈ S parallelly do
        initialize shared branch of local model θ_S^{C_i} ← θ_G
        for MTS sequence X ∈ X^i do
            reconstruct instance X as X_Rec by Eq. (6)∼Eq. (9)
            add noise and downsample X^D ← D(X + ε)
            compute loss L by Eq. (12)
            update local model:
            θ_P^{C_i} ← θ_P^{C_i} + △θ_P^{C_i}, θ_S^{C_i} ← θ_S^{C_i} + △θ_S^{C_i}
        end for
    end for
    θ_G ← Σ_{i∈S} (n_i / Σ_{i∈S} n_i) θ_S^{C_i}
end for
θ_G ← Σ_{i=1}^{m} (n_i / Σ_{i=1}^{m} n_i) θ_S^{C_i}
# Inference Stage
for client C_i, i ∈ {1, ..., m} parallelly do
    Load model parameters θ_S^{C_i} = θ_G, θ_P
    for MTS sequence X ∈ X do
        reconstruct instance X as X_Rec by Eq. (6)∼Eq. (9)
        calculate anomaly score of X: Score(X) = ||X − X_Rec||_2^2
    end for
end for
```

V. Performance Evaluation

A. Datasets

FedTADBench [8] proposes a benchmark for unsupervised entity-level MTS-AD under the framework of FL. However, it manually scatters the centralized MTS data to each client, which could not reflect the real distribution drift among clients. Moreover, it is known that the SMAP dataset suffers from the "unrealistic anomaly density" issue [47], making the experimental results on this dataset unconvincing. Furthermore, point adjust operation [48] is often applied to refining the prediction labels on entity-level MTS-AD. That is, if one or more anomalous points in an anomalous subsequence are judged as abnormal, then each anomalous time point in this subsequence is considered to be successfully detected. Unfortunately, this operation would make random guesses also perform well [49], thus impacting the comparison of different algorithms.

In this article, following previous unsupervised anomaly detection works [50], [51], we construct sequence-level unsupervised MTS-AD datasets based on three real-world distributed MTS classification datasets. For each training set with anomalies, we sample the examples of a specific class as inliers and those of the other classes as anomalies. The details of the three MTS datasets are described as follows, and their statistics are in Table I.

TABLE I
Dataset Statistics

Dataset	n	T	N	m
HHAR	13 674	500	6	5
CAP	40 390	3000	19	5
SEDFx	238 712	3000	4	4

Note: For each dataset, n is the number of samples, T is the length of time series, N is the number of variates, and m is the number of clients.

HHAR is a human activity classification dataset that contains six activities (stand, sit, walk, bike, stairs up, and stairs down). Human behavior is usually coherent, and any activity pattern change may indicate some unexpected risk (for example, if a person's activity suddenly changes from bike to stair down, he may have fallen). Therefore, unsupervised anomaly detection for human activity has practical significance. HHAR collects sensor data from five different smart devices (smartphones and smartwatches), comprising 13 674 recordings of 5 s, each of which is from six sensor channels. Since watches and cell phones are worn in different positions on the body, it is evident that there will be distribution drift among the clients.

CAP collects MTS sequences from electroencephalographic (EEG) measurements for sleep stage classification. The labels include six sleep stages (e.g., awake, non-REM 1-2-3-4, and REM) labeled by domain experts. Anomaly detection on these data could realize the sleep health monitoring of patients and provide the basis for immediate intervention treatment. The data come from five different machines and comprises 40 390 recordings of 30 s, each of which is from 19 EEG channels gathered on 41 participants.

SEDFx is also an EEG dataset for sleep stage classification with six labels similar to those of CAP. Unlike CAP, its data come from five different age groups, and EEG channels have been selected. As a whole, the dataset is composed of 238 712 recordings of 30 s, each of which is from four EEG channels gathered on 100 participants. CAP and SEDFx data are sampled 100 times per second, which causes oscillations in the MTS sequences, so we downsample them by a factor of 10 to mitigate this problem.

B. Evaluation Metrics

We use two metrics to measure the performance of anomaly detection, i.e., the area under receiver operating characteristic curve (AUC) and the average precision (AP). AUC is defined as the area under the ROC curve. In other words, we randomly select a positive sample and a negative sample from the testing set, and AUC represents the probability that the predicted value of the positive sample is greater than the negative sample. The evaluation of AUC is as follows:

$$\text{AUC} = \frac{\sum \text{pred}_{\text{pos}} > \text{pred}_{\text{neg}}}{\text{positiveNum} * \text{negativeNum}}. \tag{13}$$

TABLE II
CONFUSION MATRIX OF CLASSIFICATION

True Label	Prediction Label	
	Positive	Negative
Positive	TP	FN
Negative	FP	TN

Anomaly detection is actually binary classification. Recall and precision are often used to measure the performance of binary classification. Recall is the fraction of positive samples that are predicted correctly and precision is the fraction of positive samples that are predicted to be positive. AP represents the weighted mean of precision values at different thresholds, with the increase in recall from that of the previous threshold used as the weight. Formally, they are calculated as follows:

$$R = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{14}$$

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{15}$$

$$\text{AP} = \sum_i (R_i - R_{i-1}) P_i \tag{16}$$

where P_i and R_i are the precision and recall at the ith threshold, the meanings of TP, FN, and FP are given in the confusion matrix of Table II. We implement AUC and AP with the sklearn toolkit.

C. Compared Methods

We compare our method FDFL with five unsupervised MTS-AD methods and four commonly used FL methods. The five unsupervised MTS-AD methods include three reconstruction-based models: LSTM-AE [17], GDN [24], and USAD [52], a clustering-based method DeepSVDD [13], and a transformer-based model TranAD [21].

1) *LSTM-AE*: It utilizes LSTM to construct the encoder and decoder of AE for MTS reconstruction, the deviation between the observed and the reconstructed MTS sequences is considered as the anomaly score.
2) *DeepSVDD*: It trains a neural network while minimizing the volume of a hypersphere that encloses the network representations of the data. Finding a halfspace such that most of the data lie in and points lying outside this halfspace are deemed anomalous.
3) *USAD*: It combines the autoencoder architecture with an adversarial training framework and detects anomalies according to reconstruction loss.
4) *GDN*: It is an attention-based GNN approach that learns a graph of the dependence relationships between variates and introduces a graph deviation scoring to identify deviations from the learned relationships for MTS-AD.
5) *TranAD*: It uses self-conditioning and adversarial training to amplify error and achieve stable training, and meta-learning for anomaly detection with limited data.

The four FL mechanisms are: Fedavg [2], Fedprox [28], Scaffold [29], and Moon [30], with details as follows.

1) *Fedavg*: It iterates the local update multiple times and takes a weighted average of the local client models to obtain the global model.
2) *Fedprox*: It introduces a proximal term to help stabilize the method, which regularizes the parameters of the current local model that are not far from those of the previous global model.
3) *Scaffold*: It uses control variates to correct client-drift in its local updates, requires significantly fewer communication rounds, and is robust to data heterogeneity or client sampling.
4) *Moon*: It conducts contrastive learning between model representations to correct local training of each client, which pushes all the local models away from their previous states and pulls them toward the global model.

The implementation of these methods above can be referred to https://github.com/fanxingliu2020/FedTADBench. For FDFL and all of the compared methods, we use the same training parameters across the clients. During each training round, we randomly select 80% of the clients to update the parameters. We utilize the ADAM optimizer [53] with an initial learning rate of 10^{-4}, configure a batch size of 128, and conduct 100 rounds of global training, ultimately reporting the best results obtained. Specifically, for FDFL, we construct both the shared and private layers using a single-layer LSTM, configuring the hidden layer size to 128. All experiments are conducted in PyTorch with NVIDIA RTX 3090 24GB.

D. Experimental Results

1) Performance Comparison: Here, we present the results of the performance evaluation and compare our method with the benchmarks mentioned above. We report the AUC and AP results on each protocol (selecting samples of a specific class as normals and those from the other classes as anomalies) and the average of all protocols. Tables III, IV, and V present the results on HHAR, CAP, and SEDFx, respectively.

a) Results on HHAR: From Table III, we can see that our method FDFL performs best in most cases. The difficulty of anomaly detection on each activity is different. The activities of "Stand" and "Sit" are easier to distinguish from the other activities, where our FDFL achieves the (AUC, AP) of (0.9750, 0.6993) and (0.9591, 0.8091), respectively. The reconstruction-based models LSTM-AE, GDN, and USAD combined with any of the four FL strategies also perform well in these two activities. USAD achieves better anomaly detection performance than our method when "stand" is set as the normal class. Our method shows a significant advantage over the other methods in the other activities. The clustering-based DeepSVDD and transformer-based TranAD perform worse in all cases, which may be due to their poor representation abilities on long time series. As for the effects of different FL mechanisms on MTS-AD algorithms, Scaffold is best for LSTM-AE, DeepSVDD, and GDN, USAD achieves the best performance when combined with Moon, and TranAD prefers to FedAvg for anomaly detection on HHAR. In summary, our FDFL achieves 5.09%

TABLE III
PERFORMANCE COMPARISON ON HHAR

Fed	AD	Stand		Sit		Walk		Bike		Stair up		Stair down		AVERAGE	
		AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP
FedAvg	LSTM-AE	0.9626	0.6749	0.8580	0.2390	0.5509	0.2510	0.6038	0.2779	0.5746	0.1134	0.5235	0.1566	0.6789	0.2855
	DeepSVDD	0.6136	0.1620	0.5030	0.0927	0.5921	0.1223	0.6243	0.1250	0.4516	0.0961	0.5800	0.1244	0.5608	0.1204
	GDN	0.9718	0.7200	0.8366	0.2119	0.5738	0.2607	0.6352	0.2911	0.6771	0.2092	0.5273	0.1580	0.7036	0.3085
	USAD	0.9775	0.7111	0.8625	0.3640	0.4910	0.2210	0.6350	0.2673	0.4700	0.1201	0.3980	0.0807	0.6390	0.2940
	TranAD	0.5217	0.2802	0.7749	0.2541	0.4676	0.0875	0.5852	0.1231	0.4688	0.0838	0.4968	0.0980	0.5525	0.1545
FedProx	LSTM-AE	0.9606	0.6831	0.7469	0.1736	0.5506	0.2511	0.5985	0.2403	0.5794	0.1141	0.5227	0.1562	0.6598	0.2697
	DeepSVDD	0.6060	0.1800	0.5045	0.1039	0.5496	0.2500	0.5608	0.1372	0.3928	0.0693	0.5562	0.1635	0.5283	0.1507
	GDN	0.9765	0.6971	0.8448	0.2227	0.5853	0.2597	0.5868	0.2323	0.6679	0.1874	0.5307	0.1495	0.6987	0.2915
	USAD	0.9670	0.7321	0.3942	0.0709	0.5153	0.2781	0.5753	0.2836	0.4975	0.1306	0.4574	0.1204	0.5678	0.2664
	TranAD	0.4960	0.1889	0.7235	0.1435	0.4126	0.0981	0.4579	0.0981	0.4710	0.0858	0.5011	0.1025	0.5104	0.1195
Scaffold	LSTM-AE	0.9555	0.6724	0.9542	0.5861	0.6261	0.3270	0.5953	0.2564	0.5845	0.1304	0.5321	0.1423	0.7243	0.3524
	DeepSVDD	0.6478	0.1842	0.5030	0.0927	0.5594	0.1221	0.5793	0.1154	0.5612	0.1089	0.5800	0.1244	0.5718	0.1246
	GDN	0.9714	0.6989	0.8471	0.2318	0.5749	0.2625	0.6471	**0.2935**	0.6732	0.2624	0.5273	0.1579	0.7068	0.3178
	USAD	0.9773	0.7185	0.8349	0.4589	0.4981	0.2433	0.6212	0.2526	0.4856	0.1189	0.4010	0.0784	0.6364	0.3118
	TranAD	0.4871	0.1780	0.7489	0.1575	0.4206	0.0818	0.5570	0.1314	0.4656	0.0830	0.4968	0.0980	0.5293	0.1216
Moon	LSTM-AE	0.9502	0.6338	0.8754	0.2642	0.5509	0.2510	0.5874	0.2522	0.5746	0.1134	0.5235	0.1566	0.6770	0.2785
	DeepSVDD	0.5840	0.1368	0.5030	0.0927	0.5000	0.0906	0.6003	0.1201	0.6424	0.1318	0.5800	0.1244	0.5683	0.1161
	GDN	0.9712	0.6712	0.8217	0.2166	0.5761	0.2640	0.6242	0.1988	0.6535	0.1358	0.5275	0.1573	0.6957	0.2940
	USAD	**0.9824**	**0.7375**	0.9075	0.5285	0.4917	0.2219	0.6351	0.2700	0.4713	0.1111	0.3980	0.0808	0.6477	0.3250
	TranAD	0.5194	0.2790	0.7774	0.2567	0.4675	0.0875	0.5830	0.1231	0.4677	0.0836	0.4968	0.0980	0.5520	0.1547
FDFL		0.9750	0.6993	**0.9591**	**0.8091**	**0.6343**	**0.3616**	**0.7098**	0.2766	**0.6923**	**0.2852**	**0.5967**	**0.1890**	**0.7612**	**0.4368**

Note: HHAR contains sensor data of six human activities and separates data to clients by device type. It selects one activity as normal and collects anomalies from other activities in each protocol (proportion of anomalies/normals c is 0.1). The best results are shown in bold.

TABLE IV
PERFORMANCE COMPARISON ON CAP

Fed	AD	Awake		Non-REM1		Non-REM2		Non-REM3		Non-REM4		REM		AVERAGE	
		AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP
FedAvg	LSTM-AE	0.4671	0.0821	0.5397	0.1034	0.5221	0.1025	0.5050	0.0976	0.4511	0.1006	0.6394	0.1939	0.5207	0.1134
	DeepSVDD	0.5039	0.0919	0.5021	0.1003	0.4999	0.0906	0.5056	0.0918	0.5000	0.0910	0.5000	0.0909	0.5019	0.0928
	GDN	0.4874	0.0872	0.5283	0.1041	0.5121	0.0994	**0.5149**	0.1013	0.5071	0.1015	0.6108	0.1333	0.5268	0.1045
	USAD	0.4572	0.0787	0.5206	0.0989	0.5019	0.0977	0.5052	0.0961	0.4438	0.0986	0.6072	0.1341	0.5060	0.1007
	TranAD	0.5026	0.0870	0.5032	0.0919	0.4999	0.0914	0.4979	0.0917	0.4948	0.0896	0.5051	0.1043	0.5006	0.0927
FedProx	LSTM-AE	0.4610	0.0818	0.5418	0.1013	0.5141	0.1016	0.5045	0.0973	0.4516	0.1003	0.6657	0.2150	0.5231	0.1162
	DeepSVDD	0.5000	0.0910	0.5022	0.1007	0.5000	0.0910	0.5019	0.0911	0.5000	0.0910	0.5000	0.0909	0.5007	0.0926
	GDN	0.5173	0.0975	0.5291	0.1057	0.5116	0.0932	0.5105	0.1023	0.4843	0.0927	0.6235	0.1678	0.5294	0.1099
	USAD	0.4539	0.0782	0.5191	0.0991	0.5044	0.0972	0.4983	0.0940	0.4510	0.0978	0.6353	0.1618	0.5103	0.1047
	TranAD	0.5017	0.0898	0.4965	0.0907	0.5001	0.0918	0.4979	0.0915	0.4966	0.0887	0.5025	0.1053	0.4992	0.0930
Scaffold	LSTM-AE	0.4766	0.0828	0.5334	0.1034	0.5323	0.1034	0.5038	0.0951	0.4583	0.0861	0.6153	0.1219	0.5200	0.0988
	DeepSVDD	0.5039	0.0919	0.5021	0.1003	0.5000	0.0910	0.5056	0.0918	0.5000	0.0910	0.5000	0.0909	0.5019	0.0928
	GDN	0.5011	0.0903	0.5264	0.1084	0.5107	0.1015	0.4999	0.0952	0.4839	0.0902	0.6101	0.1515	0.5220	0.1062
	USAD	0.4638	0.0798	0.5205	0.0989	0.5008	0.0949	0.5026	0.0936	0.4443	0.0935	0.6445	0.1782	0.5128	0.1065
	TranAD	0.5040	0.0874	0.5018	0.0930	0.4999	0.0914	0.4979	0.0917	0.4948	0.0896	0.5050	0.1031	0.5006	0.0927
Moon	LSTM-AE	0.4671	0.0821	0.5347	0.1045	0.5194	0.1018	0.5038	0.0963	0.4485	0.1014	0.6368	0.1873	0.5286	0.1122
	DeepSVDD	0.5039	0.0919	0.5021	0.1003	0.5003	0.0916	0.5056	0.0918	0.5000	0.0910	0.5000	0.0909	0.5020	0.0929
	GDN	0.5156	0.0939	0.5081	0.1033	0.5104	0.0951	0.5071	0.0918	0.4996	0.0979	0.5789	0.1147	0.5200	0.0995
	USAD	0.4574	0.0787	0.5198	0.0986	0.5015	0.0979	0.5052	0.0961	0.4436	0.0985	0.6061	0.1331	0.5056	0.1005
	TranAD	0.5026	0.0870	0.5032	0.0920	0.4999	0.0914	0.4979	0.0917	0.4948	0.0896	0.5050	0.1042	0.5006	0.0927
FDFL		**0.5460**	**0.1145**	**0.5492**	**0.1120**	**0.5566**	**0.1099**	0.5062	**0.1121**	**0.5401**	**0.1040**	**0.6731**	**0.2448**	**0.5618**	**0.1329**

Note: CAP contains EEG data of six sleep stages and separates data to clients by machine ID. It selects one sleep stage as normal and collects anomalies from other sleep stages in each protocol (proportion of anomalies/normals c is 0.1). The best results are shown in bold.

and 24.0% improvement on average AUC and AP over the best benchmark method.

b) Results on CAP: Table IV shows the anomaly detection performance on the EEG dataset CAP. In the "REM" sleep stage, human eyes twitch, and the brain is even more active than in waking hours. It is more distinguishable than the other sleep stages, where our FDFL achieves the (AUC, AP) of (0.6731, 0.2448). Anomaly detection performance in the other sleep stages is poor, especially in "None-REM3," which is more easily confused with other sleep stages, and only has a maximum AUC of 0.5149 obtained by Fedavg-GDN. GDN-based methods get performance second only to our FDFL because GDN models the relationship between variables by graph, which is more suitable for the MTS data with more variables.

TABLE V
PERFORMANCE COMPARISON ON SEDFX

Fed	AD	Awake		Non-REM1		Non-REM2		Non-REM3		Non-REM4		REM		AVERAGE	
		AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP	AUC	AP
FedAvg	LSTM-AE	0.4507	0.0856	0.5575	0.1021	0.5984	0.2785	0.4446	0.1119	0.3499	0.0668	0.5607	0.1837	0.4936	0.1384
	DeepSVDD	0.4985	0.0888	0.5077	0.0935	0.5109	0.1144	0.4982	0.0911	0.4986	0.0900	0.5114	0.0978	0.5042	0.0959
	GDN	0.4891	0.0865	0.5454	**0.1197**	0.5623	0.1574	0.5003	0.0958	0.4883	0.0834	0.5430	0.1192	0.5214	0.1103
	USAD	0.4300	0.0758	0.5609	0.1063	0.5628	0.2008	0.3781	0.1000	0.3509	0.0696	0.5546	0.1388	0.4729	0.1152
	TranAD	0.5104	0.0901	0.5135	0.0930	0.5070	0.0939	0.5050	0.0907	0.5021	0.0982	0.5228	0.0987	0.5101	0.0941
FedProx	LSTM-AE	0.4459	0.0828	0.5482	0.1006	0.5961	0.2626	0.4277	0.1063	0.4224	0.0804	0.5602	0.1738	0.5001	0.1344
	DeepSVDD	0.4990	0.0891	0.5122	0.0936	0.5135	0.1195	0.5038	0.0900	0.4980	0.0886	0.5118	0.0965	0.5064	0.0962
	GDN	0.5046	0.0883	0.5438	0.1014	0.5416	0.1558	0.4996	0.0897	0.4959	0.0859	0.5496	0.1216	0.5225	0.1071
	USAD	0.4298	0.0758	0.5570	0.1052	0.5542	0.1870	0.3928	0.1083	0.3389	0.0690	0.5565	0.1394	0.4715	0.1141
	TranAD	0.5106	0.0888	0.5142	0.1109	0.5118	0.0947	0.5058	0.0924	0.5089	0.0921	0.5157	0.0987	0.5112	0.0963
Scaffold	LSTM-AE	0.4614	0.0833	0.5575	0.1021	0.5800	0.2673	0.4446	**0.1119**	0.3523	0.0800	0.5971	0.1799	0.5281	0.1374
	DeepSVDD	0.4985	0.0887	0.5064	0.1033	0.5097	0.1155	0.4947	0.0922	0.4966	0.0896	0.5098	0.0964	0.5026	0.0976
	GDN	0.4971	0.0876	0.5536	0.1182	0.5633	0.1411	0.5113	0.0963	0.4960	0.0866	0.5618	0.1295	0.5305	0.1099
	USAD	0.4303	0.0758	0.5610	0.1063	0.5690	0.1897	0.4038	0.0941	0.3505	0.0776	0.5680	0.1304	0.4804	0.1123
	TranAD	0.5109	0.0901	0.5163	0.0940	0.5108	0.0958	0.5050	0.0907	0.5055	0.0923	0.5220	0.0985	0.5118	0.0936
Moon	LSTM-AE	0.4538	0.0857	0.5575	0.1021	0.6000	0.2680	0.4446	0.1119	0.3499	0.0668	0.5602	0.1843	0.4946	0.1365
	DeepSVDD	0.5034	0.0892	0.5116	0.1052	0.5047	0.1060	0.5032	0.0888	0.5115	0.0943	0.5122	0.1049	0.5078	0.0981
	GDN	**0.5184**	0.0925	0.5472	0.1017	0.5731	0.2073	0.4852	0.0980	0.5004	0.0917	0.5520	0.1243	0.5294	0.1193
	USAD	0.4300	0.0758	0.5610	0.1063	0.5911	0.2591	0.3744	0.1010	0.3471	0.0697	0.5546	0.1388	0.4764	0.1251
	TranAD	0.5104	0.0901	0.5137	0.0945	0.5070	0.0939	0.5050	0.0907	0.5021	0.0982	0.5231	0.0987	0.5102	0.0944
FDFL		0.5176	**0.0964**	**0.5696**	0.1191	**0.6304**	**0.2830**	**0.5168**	0.1104	**0.5226**	**0.1218**	**0.6118**	**0.2023**	**0.5615**	**0.1555**

Note: SEDFx contains EEG of six sleep stages as CAP, but it separates data by age group. It selects one sleep stage as normal and collects anomalies from other sleep stages in each protocol (proportion of anomalies/normals c is 0.1). The best results are shown in bold.

As for the FL effect, DeepSVDD and TranAD are not affected by the difference in federated mechanism, they perform worse with any federated strategies because of their limited abilities to represent MTS data. GDN achieves the best performance when combined with Fedprox, LSTM-AE and USAD fit best in Moon and Scaffold, respectively. Our method achieves 6.12% and 14.37% improvement on average AUC and AP over the best benchmark method.

c) Results on SEDFx: SEDFx is also a decentralized EEG MTS dataset like CAP, with fewer variables, but it splits the data into different clients according to user age rather than machine id. As shown in Table V, the anomaly detection performance in "None-REM2" and "REM" is better than that in the other sleep stages. Similar to the results on CAP, GDN performs better than the other benchmark methods. DeepSVDD and TranAD consistently perform poorly, and LSTM-AE, GDN, and USAD fit best in Moon, Fedprox, and Scaffold, respectively. Our method achieves 5.84% and 12.36% improvement on average AUC and AP over the best benchmark method.

Furthermore, for MTS-AD methods, reconstruction-based methods LSTM-AE, GDN, and USAD generally have better performance than the clustering-based method DeepSVDD and the transformer-based method TranAD, due to their better representation abilities for long time series. The performance of FL mechanisms is inconsistent across different datasets or anomaly detection methods. They are less effective on DeepSVDD and TranAD, and LSTM-AE, GDN, and USAD perform well in combination with Moon, FedProx, and Scaffold, respectively. Our FDFL achieves the best anomaly detection performance in most cases.

2) Ablation Study: In this section, we perform ablation study to demonstrate the effectiveness of three modules of our model: *self attention* (SA), *heterogeneity robust loss* ($\mathcal{L}_{\text{HRL}}$), and *personalized loss* ($\mathcal{L}_{\text{PL}}$). The results are presented in Table VI. We can see that the performance on the three datasets shows almost the same pattern, except for the SA module on the CAP dataset. The introduction of SA leads to a decline in performance on CAP, probably because it has more variables and attention is difficult to train. Generally, taking the AUC on HHAR as an example, after using only $\mathcal{L}_{\text{HRL}}$, $\mathcal{L}_{\text{PL}}$, or SA, the result is improved by 3.56%, 3.95%, and 1.52%, respectively. When using both $\mathcal{L}_{\text{HRL}}$ and $\mathcal{L}_{\text{PL}}$, the performance gets 6.91% lift over the baseline. This suggests that both $\mathcal{L}_{\text{HRL}}$ and $\mathcal{L}_{\text{PL}}$ can boost the representation ability on MTS data, while SA can further improve FDFL performance.

TABLE VI
ABLATION STUDY OF FDFL

$\mathcal{L}_{\text{HRL}}$	$\mathcal{L}_{\text{PL}}$	SA	HHAR		CAP		SEDFx	
			AUC	AP	AUC	AP	AUC	AP
×	×	×	0.7019	0.3601	0.5201	0.0920	0.5101	0.1344
✓	×	×	0.7269	0.3919	0.5378	0.1168	0.5278	0.1385
×	✓	×	0.7296	0.3998	0.5346	0.1244	0.5318	0.1369
×	×	✓	0.7126	0.3915	0.5178	0.1020	0.5268	0.1411
✓	✓	×	0.7504	0.4222	**0.5630**	**0.1380**	0.5468	0.1480
×	✓	✓	0.7407	0.3978	0.5439	0.1131	0.5365	0.1428
✓	×	✓	0.7437	0.4029	0.5536	0.1194	0.5481	0.1451
✓	✓	✓	**0.7612**	**0.4368**	0.5618	0.1329	**0.5615**	**0.1555**

Note: $\mathcal{L}_{\text{HRL}}$ represents Heterogeneity Robust Loss; $\mathcal{L}_{\text{PL}}$ indicates Personalized Loss; and SA is the self-attention module.

3) Effects of Hyperparameters α and β: The hyperparameters α and β represent the contributions of the heterogeneity robust loss $\mathcal{L}_{\text{HAR}}$ and personalized loss $\mathcal{L}_{\text{PL}}$ to federated unsupervised MTS-AD. We explore the impacts of α and β on the anomaly detection performance of FDFL. We perform grid

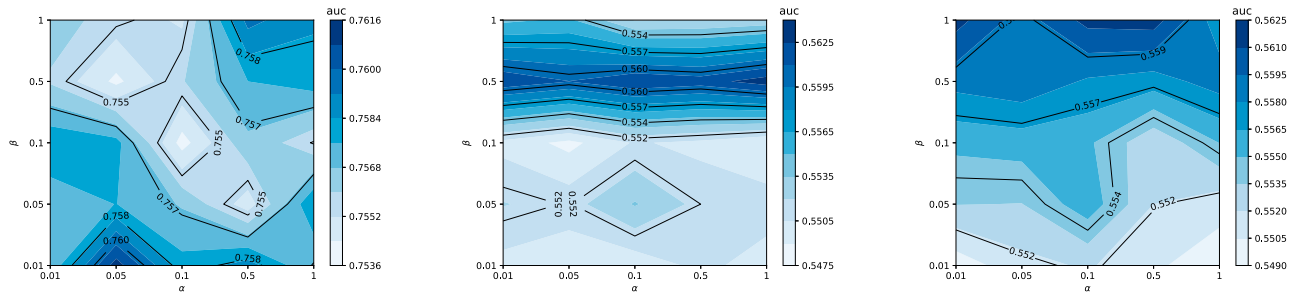

Fig. 3. Performance (AUC) versus α and β. (a) AUC on HHAR. (b) AUC on CAP. (c) AUC on SEDFx.

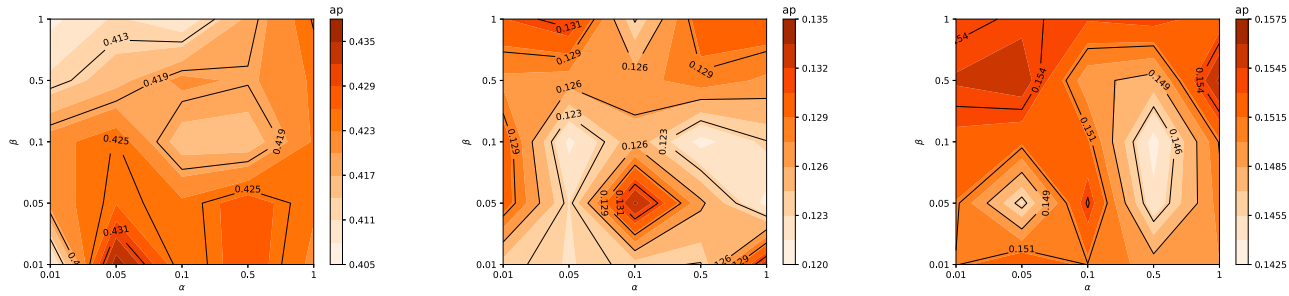

Fig. 4. Performance (AP) versus α and β. (a) AP on HHAR. (b) AP on CAP. (c) AP on SEDFx.

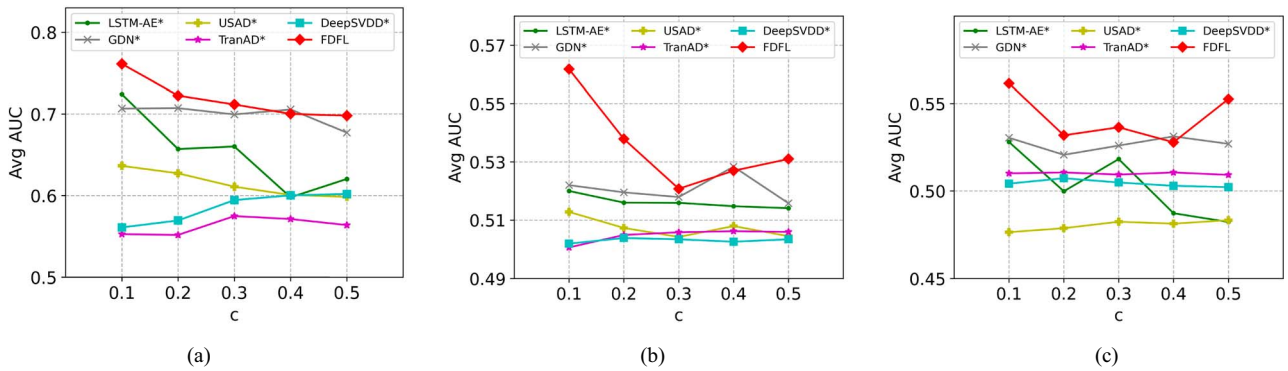

Fig. 5. Performance (AUC) versus c (increasing from 0.1 to 0.5). (a) HHAR. (b) CAP. (c) SEDFx.

search from $\{0.01, 0.05, 0.1, 0.5, 1\}$ for α and β, the experiment results of average AUC and average AP are shown in Figs. 3 and 4, respectively. Different data have different heterogeneity and personalization, so their optimal hyperparameter values are different. On the HHAR dataset, the average AUC and average AP reach the best when $\alpha = 0.05$ and $\beta = 0.01$. On the SEDFx dataset, $\alpha = 0.01$ and $\beta = 0.5$ result in the best average AUC and average AP. On the CAP dataset, α has little impact on average AUC, and a beta of 0.5 is better for AUC performance, while $\alpha = 0.5$ and $\beta = 0.05$ are best for AP performance. Acceptable AUC and AP can be achieved when $\alpha = 0.01$ and $\beta = 0.5$. As a result, we set $\alpha = 0.05$ and $\beta = 0.01$ on HHAR, and $\alpha = 0.01$ and $\beta = 0.5$ on CAP and SEDFx. This is because the differences between distributed EEG data (CAP and SEDFx) are greater than those between distributed human activity data (HHAR), and thus a larger PL is required to enhance the complementarity of the private branch.

4) Effect of Anomaly Proportion c: c is the proportion of anomalies/normals in the training data. To explore the impact of c on our FDFL, we change it from 0.1 to 0.5. Figs. 5 and 6 show the AUC and AP results on HHAR, CAP, and SEDFx, where $*$ indicates that the MTS-AD method works with the best-federated mechanism. We can see that as c increases, AUC shows a decreasing trend, while AP has an increasing trend. With more anomalies in the training set, the normal pattern is more challenging to model, which increases the probability that anomalies are judged as normals.

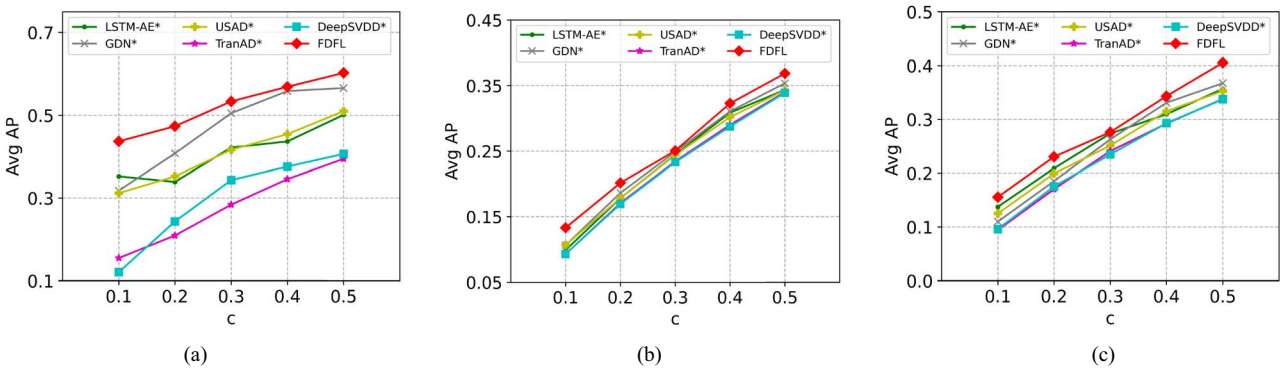

Fig. 6. Performance (AP) versus c (increasing from 0.1 to 0.5). (a) HHAR. (b) CAP. (c) SEDFx.

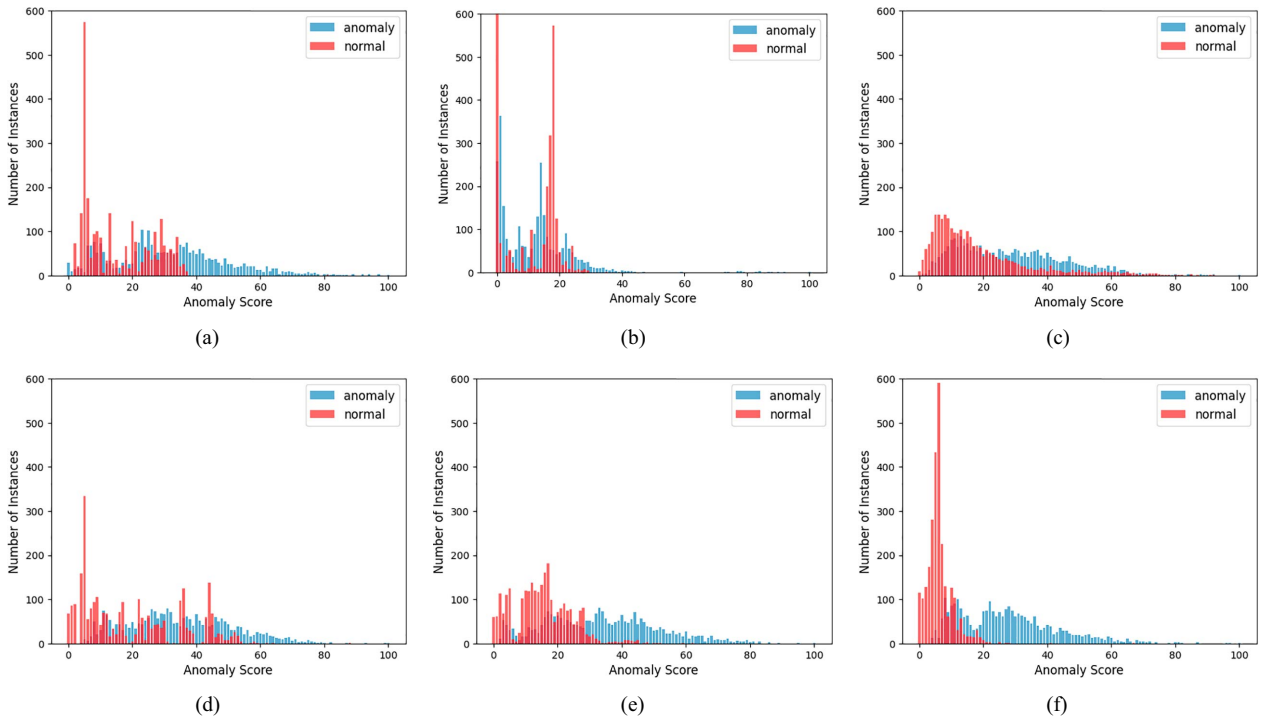

Fig. 7. Anomaly score histograms of anomalies (blue) and normals (red) for different methods on HHAR. It visualizes the results on "sit" class and sets the proportion of anomalies/normals c as 0.5. (a) LSTM-AE*. (b) DEEPSVDD*. (c) GDN*. (d) USAD*. (e) TranAD*. (f) FDFL.

Furthermore, since we only increase the anomalies while the number of normals is fixed, this leads to the increase of AP. The reconstruction-based methods are superior to the clustering-based and transformer-based methods. The GDN-based approaches achieve the best performance among the baselines, which benefits from effectively characterizing the variate relationship by its GNN. Our FDFL introduces feature decoupling to mitigate the effects of noise and heterogeneity while maintaining the unique characteristics of clients, thus improving the representations of MTS data on each client. As shown in Figs. 5 and 6, FDFL significantly outperforms the others on the three MTS datasets.

5) Anomaly Score Distributions of Inliers and Outliers: To check whether or not federated MTS-AD methods can distinguish normals and anomalies, we calculate the anomaly scores of testing samples for each method and visualize the accumulated histograms for normals and anomalies, respectively. The anomaly score of DeepSVDD is the distance of the MTS representation to the cluster center, while the other methods take the reconstruction error as the anomaly score. We take the "sit" class of the HHAR dataset as an example, and the visualization results are shown in Fig. 7. Most normals have smaller anomaly scores, while most anomalies have larger ones. The DeepSVDD-based and USAD-based methods can obtain small anomaly scores for normals, but they also have small anomaly scores for anomalies, making the anomalies challenging to distinguish. The anomaly score distributions of GDN and FDFL conform to a Gaussian distribution, which shows their stability. The difference lies in that compared with GDN, FDFL has a smaller reconstruction

error of normals, which makes it perform better in detecting anomalies.

VI. Conclusion and Future Works

This article presents a novel FL framework incorporating a dual-stream feature decoupling network designed for sequence-level anomaly detection in decentralized MTS data. The proposed method FDFL decouples the representation network into two branches: a shared branch and a private branch, and only aggregates the parameters of the shared branch among clients while keeping the private branches personalized. To guide feature decoupling, we introduce a contrastive loss function. This loss generates positive instances through noise addition and downsampling while using the distance between the representations of the shared and private branches as a negative component to enforce orthogonality. In such a way, the shared branch focuses on representing common patterns and is robust to noise and heterogeneity, while the private branches are a complement to the shared branch to better fit the local data. Extensive experiments on three MTS datasets show that FDFL outperforms the benchmark methods in most cases.

For future work, first we will investigate integrating our MTS feature decoupling approach with various neural network architectures to assess its broader applicability. Second, in FDFL we apply a simple MTS data augmentation method to construct positive pairs for contrastive learning, which assumes that frequency does not affect representations for anomaly detection. This assumption holds for human activities and EEG data but lacks adaptability. To handle this issue, we will try to find more appropriate approaches to obtain the positive pairs for training the shared branch in our method. Last but not least, this work is generally a deep learning-based method of unsupervised MTS anomaly detection in FL scenarios, which inevitably inherits the black-box nature of deep learning models. In the future, we will try to conduct an investigation on its interpretability and theoretical analysis on its optimization stability and performance guarantee under different parameter settings and data distributions.

References

[1] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol. (TIST)*, vol. 10, no. 2, pp. 1–19, 2019.

[2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, PMLR, 2017, pp. 1273–1282.

[3] C. I. Bercea, B. Wiestler, D. Rueckert, and S. Albarqouni, "Federated disentangled representation learning for unsupervised brain anomaly detection," *Nature Mach. Intell.*, vol. 4, no. 8, pp. 685–695, 2022.

[4] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, "Federated learning for Internet of Things: A comprehensive survey," *IEEE Commun. Surveys Tut.*, vol. 23, no. 3, pp. 1622–1658, thirdquart. 2021.

[5] F. Cavallin and R. Mayer, "Anomaly detection from distributed data sources via federated learning," in *Proc. 36th Int. Conf. Adv. Inf. Netw. Appl. (AINA)*, vol. 2, Cham, Switzerland: Springer International Publishing, 2022, pp. 317–328.

[6] V. Mothukuri, P. Khare, R. M. Parizi, S. Pouriyeh, A. Dehghantanha, and G. Srivastava, "Federated-learning-based anomaly detection for IoT security attacks," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2545–2554, Feb. 2022.

[7] B. Li, Y. Wu, J. Song, R. Lu, T. Li, and L. Zhao, "DeepFed: Federated deep learning for intrusion detection in industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 17, no. 8, pp. 5615–5624, Aug. 2021.

[8] F. Liu et al., "FedTADBench: Federated time-series anomaly detection benchmark," 2022, *arXiv:2212.09518*.

[9] M. M. Breunig, H. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, Dallas, TX, USA, W. Chen, J. F. Naughton, and P. A. Bernstein, Eds., New York, NY, USA: ACM, May 16–18, 2000, pp. 93–104. [Online]. Available: https://doi.org/10.1145/342009.335388

[10] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proc. 6th Int. Conf. Learn. Represent. (ICLR)*, Vancouver, BC, Canada, *OpenReview.net*, April 30–May 3, 2018, pp. 1–19. [Online]. Available: https://openreview.net/forum?id=BJJLHbb0-

[11] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001. [Online]. Available: https://doi.org/10.1162/089976601750264965

[12] D. M. Tax and R. P. Duin, "Support vector data description," *Mach. Learn.*, vol. 54, no. 1, pp. 45–66, 2004.

[13] L. Ruff et al., "Deep one-class classification," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 4393–4402.

[14] L. Shen, Z. Li, and J. T. Kwok, "Timeseries anomaly detection using temporal hierarchical one-class network," in *Proc. Adv. Neural Inf. Process. Syst. 33: Annu. Conf. Neural Inf. Process. Syst. (NeurIPS)*, virtual, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., Dec. 6–12, 2020, pp. 13016–13026. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/97e401a02082021fd24957f852e0e475-Abstract.html

[15] J. Lin, Y. He, W. Xu, J. Guan, J. Zhang, and S. Zhou, "Latent feature reconstruction for unsupervised anomaly detection," *Appl. Intell.*, vol. 53, no. 20, pp. 23628–23640, 2023.

[16] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *Proc. 25th Int. Conf. Mach. Learn.*, 2008, pp. 1096–1103.

[17] D. Park, Y. Hoshi, and C. C. Kemp, "A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder," *IEEE Robot. Autom. Lett.*, vol. 3, no. 2, pp. 1544–1551, Jul. 2018. [Online]. Available: https://doi.org/10.1109/LRA.2018.2801475

[18] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, Anchorage, AK, USA, A. Teredesai, V. Kumar, Y. Li, R. Rosales, E. Terzi, and G. Karypis, Eds., New York, NY, USA: ACM, Aug. 14—18, 2019, pp. 2828–2837. [Online]. Available: https://doi.org/10.1145/3292500.3330672

[19] Z. Li et al., "Multivariate time series anomaly detection and interpretation using hierarchical inter-metric and temporal embedding," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining (KDD)*, Virtual Event, Singapore, F. Zhu, B. C. Ooi, and C. Miao, Eds., New York, NY, USA: ACM, Aug. 14–18, 2021, pp. 3220–3230. [Online]. Available: https://doi.org/10.1145/3447548.3467075

[20] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.

[21] S. Tuli, G. Casale, and N. R. Jennings, "Tranad: deep transformer networks for anomaly detection in multivariate time series data," *Proc. VLDB Endowment*, vol. 15, no. 6, pp. 1201–1214, 2022.

[22] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly transformer: Time series anomaly detection with association discrepancy," 2021, *arXiv:2110.02642*.

[23] Y. He et al., "Variate associated domain adaptation for unsupervised multivariate time series anomaly detection," *ACM Trans. Knowl. Discovery Data*, vol. 18, no. 8, 2024, Art. no. 187.

[24] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. 35th AAAI Conf. Artif. Intell. (AAAI) -33rd Conf. Innovative Appl. Artif. Intell. (IAAI), 11th Symp. Educational Adv. Artif. Intell. (EAAI)*, Virtual Event, Palo Alto, CA, USA: AAAI Press, Feb. 2–9, 2021, pp. 4027–4035. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/16523

[25] W. Chen, L. Tian, B. Chen, L. Dai, Z. Duan, and M. Zhou, "Deep variational graph convolutional recurrent network for multivariate time series anomaly detection," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2022, pp. 3621–3633.

[26] S. Schmidl, P. Wenig, and T. Papenbrock, "Anomaly detection in time series: A comprehensive evaluation," *Proc. VLDB Endowment*, vol. 15, no. 9, pp. 1779–1797, 2022.

[27] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends® Mach. Learn.*, vol. 14, nos. 1–2, pp. 1–210, 2021.

[28] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proc. Mach. Learn. Syst.*, vol. 2, pp. 429–450, 2020.

[29] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2020, pp. 5132–5143.

[30] Q. Li, B. He, and D. Song, "Model-contrastive federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis, Pattern Recognit.*, 2021, pp. 10713–10722.

[31] N. Wu, N. Zhang, W. Wang, L. Fan, and Q. Yang, "Fadman: Federated anomaly detection across multiple attributed networks," 2022, *arXiv:2205.14196*.

[32] Z. Wang et al., "FederatedScope-GNN: Towards a unified, comprehensive and efficient package for federated graph learning," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2022, pp. 4110–4120.

[33] K. Zhang, Y. Jiang, L. Seversky, C. Xu, D. Liu, and H. Song, "Federated variational learning for anomaly detection in multivariate time series," in *Proc. IEEE Int. Perform., Comput., Commun. Conf. (IPCCC)*, Piscataway, NJ, USA: IEEE, 2021, pp. 1–9.

[34] Y. Liu et al., "Deep anomaly detection for time-series data in industrial IoT: A communication-efficient on-device federated learning approach," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6348–6358, Apr. 2021.

[35] J. Yang, S. E. Reed, M.-H. Yang, and H. Lee, "Weakly-supervised disentangling with recurrent transformations for 3d view synthesis," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 1099–1107.

[36] X. Ma, X. Kong, S. Zhang, and E. H. Hovy, "Decoupling global and local representations via invertible generative flows," in *Proc. Int. Conf. Learn. Represent*, pp. 1–36.

[37] W.-N. Hsu, Y. Zhang, and J. Glass, "Unsupervised learning of disentangled and interpretable representations from sequential data," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1878–1889.

[38] J. Ma, C. Zhou, P. Cui, H. Yang, and W. Zhu, "Learning disentangled representations for recommendation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 5712–5723.

[39] M. F. Mathieu, J. J. Zhao, J. Zhao, A. Ramesh, P. Sprechmann, and Y. LeCun, "Disentangling factors of variation in deep representation using adversarial training," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 5047–5055.

[40] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2013, *arXiv:1312.6114*.

[41] I. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger, Eds., vol. 27, Cambridge, MA, USA: MIT Press, Inc., 2014. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2014/file/5ca3e9b122f61f8f06494c97b1afccf3-Paper.pdf

[42] L. Yingzhen and S. Mandt, "Disentangled sequential autoencoder," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 5670–5679.

[43] H. Kim and A. Mnih, "Disentangling by factorising," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 2649–2658.

[44] Y. Zhu, M. R. Min, A. Kadav, and H. P. Graf, "S3vae: Self-supervised sequential vae for representation disentanglement and data generation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 6538–6547.

[45] S. Tonekaboni, C.-L. Li, S. O. Arik, A. Goldenberg, and T. Pfister, "Decoupling local and global representations of time series," in *Proc. Int. Conf. Artif. Intell. Statist.*, PMLR, 2022, pp. 8700–8714.

[46] C. Wang, S. Xing, R. Gao, L. Yan, N. Xiong, and R. Wang, "Disentangled dynamic deviation transformer networks for multivariate time series anomaly detection," *Sensors*, vol. 23, no. 3, p. 1104, 2023.

[47] R. Wu and E. J. Keogh, "Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 3, pp. 2421–2429, Mar. 2023.

[48] H. Xu et al., "Unsupervised anomaly detection via variational auto-encoder for seasonal kpis in web applications," in *Proc. World Wide Web Conf.*, 2018, pp. 187–196.

[49] S. Kim, K. Choi, H.-S. Choi, B. Lee, and S. Yoon, "Towards a rigorous evaluation of time-series anomaly detection," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 7, 2022, pp. 7194–7201.

[50] C. Zhou and R. C. Paffenroth, "Anomaly detection with robust deep autoencoders," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2017, pp. 665–674.

[51] W.-Y. Lin, Z. Liu, and S. Liu, "Locally varying distance transform for unsupervised visual anomaly detection," in *Proc. 17th Eur. Conf. Comput. Vis. (ECCV)*, Tel Aviv, Israel, Part XXX. Cham, Switzerland: Springer Nature Switzerland, Oct. 23–27, 2022, pp. 354–371.

[52] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "Usad: Unsupervised anomaly detection on multivariate time series," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2020, pp. 3395–3404.

[53] D. P. Kingma and J. L. Ba, "Adam: A method for stochastic optimization," in *3rd Int. Conf. Learn. Represent. (ICLR)*, San Diego, CA, USA, May 7–9, 2015, pp. 1–15.